

Four Architectures for Peripheral Blood Cell Classification

A Comparative Study on BloodMNIST

Jelena Nikolic[†]

Abstract—Automated classification of peripheral blood cell types from microscopic images can support faster and more consistent hematological diagnosis than manual analysis. In this work, eight blood cell types from the BloodMNIST benchmark were classified by comparing four different deep learning approaches: a baseline convolutional neural network (CNN), one with Squeeze-and-Excitation channel attention, a patch-based decision fusion CNN, and a Spiking Neural Network (SNN), implemented in PyTorch as required by the course rules. On a held-out test set of 3,421 images, the tuned attention-based CNN achieved the highest accuracy (96.99%), closely followed by the patch-based fusion model (96.87%) and the center-crop ablation (96.84%); the baseline CNN reached 91.58%. The SNN achieved 87.26% accuracy while using roughly half as many parameters as the CNN variants. An additional ablation experiment tested whether a single center crop of the input could match patch-based fusion without its extra training cost. Results varied across training runs, showing that when two architectures are this close in accuracy, a single run is not enough to tell which mechanism explains the difference.

Index Terms—Blood cell classification, convolutional neural networks, channel attention, spiking neural networks, medical image classification.

I. INTRODUCTION

Peripheral blood smear analysis is a routine diagnostic procedure. It is used to detect infections, anemia, leukemia, and other blood conditions. Expert hematologists classify blood cells manually, and this is accurate. However, it takes time, and different doctors can disagree on the same case. This has motivated growing interest in automated, image-based classification systems as a support tool for clinical work.

Previous studies on automated blood cell classification have mostly used a single convolutional architecture, tested on private or small datasets [1]. This makes it hard to reproduce results or compare different studies. BloodMNIST, part of the standardized MedMNIST v2 benchmark [2], addressed this problem: it provides a public, pre-processed dataset with official train, validation, and test splits. However, the benchmark mainly tests generic methods, such as ResNet and AutoML, across eighteen different datasets. It does not study architecture-specific mechanisms, such as channel attention, or input representations designed specifically for blood cell classification. Squeeze-and-Excitation attention [3] has been shown to improve image classification in other settings, but its effect on small, moderately imbalanced medical datasets, and on training stability, has received less attention.

This work focuses on the classification of eight peripheral blood cell types from microscopic images using the BloodMNIST benchmark. Automated blood cell classification is an important research topic due to the growing use of AI-assisted methods in hematological diagnosis, where reducing manual analysis time and inter-observer variability can provide practical clinical benefits. Four different architectures are implemented and compared: a baseline CNN, a CNN with Squeeze-and-Excitation channel attention, a patch-based decision fusion CNN, and a Spiking Neural Network. This comparison allows us to evaluate which architectural choices contribute the most to the classification performance in this task. While previous work on BloodMNIST has mainly focused on single architectures, such as ResNet [2], an additional ablation study is performed to test one possible explanation for patch-based fusion’s advantage over the baseline. The results differ across training runs when the two models are close in accuracy, showing that a single run is not always enough to draw a firm conclusion about which mechanism is responsible. Training behaviour was also analyzed in detail: the attention-based CNN showed clear instability in its early training epochs, and a similar problem was identified in an earlier session with the Spiking Neural Network at higher input resolution, providing practical insights for the use of different architecture families on small and moderately imbalanced medical image datasets.

The main contributions of this work are:

- 1) Four architecturally distinct models are designed and trained on BloodMNIST: a baseline CNN, an attention-based CNN, a patch-fusion CNN, and a Spiking Neural Network.
- 2) Two independent training instabilities are identified and corrected: one in the attention-based CNN and one in the Spiking Neural Network when using higher-resolution inputs. The causes of these instabilities and their effects on the final classification accuracy are also analyzed.
- 3) A center-crop ablation study tests one possible explanation for patch-based fusion’s advantage over the baseline. The results differ across training runs when the two models are close in accuracy, showing that a single run is not always enough to draw a firm conclusion about which mechanism is responsible.
- 4) Model complexity and computational efficiency are analyzed in addition to classification accuracy, considering factors such as the number of parameters and training time.

The rest of this report is organized as follows. Sec. II reviews related work. Sec. III presents the overall processing

[†]Department of Mathematics, University of Padova

[‡] Jelena Nikolic, email: jelena.nikolic@studenti.unipd.it

pipeline. Sec. IV describes the dataset and preprocessing. Sec. V details the four learning architectures. Sec. VI presents and discusses the results. Sec. VII concludes the report.

II. RELATED WORK

The dataset used in this work comes from Acevedo et al. [1]. It has 17,092 expert-annotated microscopic images from eight peripheral blood cell types, acquired with a CellaVision DM96 analyzer at the Hospital Clinic of Barcelona. This large, high-quality, public dataset enabled further research on automated blood cell classification. However, the original data article does not test different classification architectures. It only refers to one companion CNN study [4]. So the effect of different architectural choices on this dataset is still mostly unexplored.

Yang et al. [2] later added this dataset to MedMNIST v2, a standardized benchmark of eighteen 2D and 3D biomedical image datasets with official train, validation, and test splits. This improved reproducibility: it defines one common preprocessing pipeline (center-crop to 200x200 pixels, then resize to 28, 64, 128, or 224 pixels) and reports baseline results using ResNet-18, ResNet-50, and AutoML methods. For BloodMNIST, ResNet-18 at 28x28 resolution reached 95.8% accuracy and an AUC of 0.998. This standardization makes it easier to compare studies. However, the evaluation is meant to be general across all eighteen datasets, so it does not look at task-specific mechanisms, such as channel attention, different spatial input representations, or different computing paradigms. This work instead studies BloodMNIST in more depth: it compares four architecturally different approaches and directly analyzes training stability issues.

Squeeze-and-Excitation networks introduced a lightweight channel-attention mechanism. It improved ImageNet classification accuracy across several CNN architectures, with only a small increase in parameters [3]. Since then, Squeeze-and-Excitation blocks have been widely used to improve convolutional architectures. But the original evaluation used ImageNet, a dataset with over one million training images and heavily tuned training procedures. How this mechanism behaves on much smaller, moderately imbalanced medical datasets has received less attention. In this work, the Squeeze-and-Excitation CNN was less stable during training than the baseline CNN, under the same starting hyperparameters. A lower learning rate and longer early-stopping patience were needed to reach its best performance (Sec. VI). This is a task-specific finding that adds to what the original, large-scale study reported.

III. PROCESSING PIPELINE

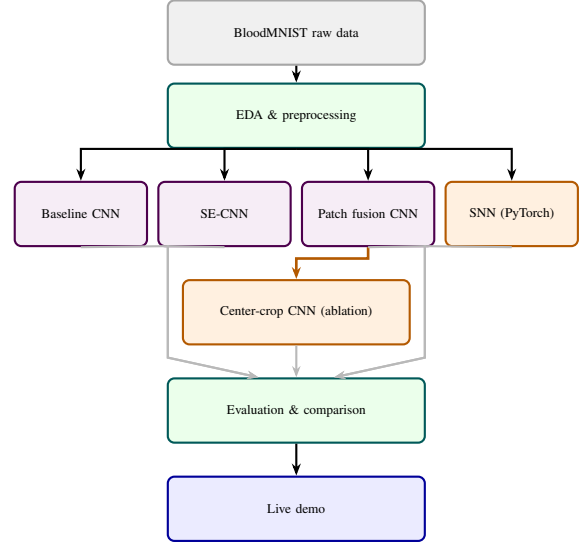


Fig. 1: Overview of the processing pipeline, from raw data to live demo.

Fig.1 illustrates the overall processing pipeline. Raw BloodMNIST images are first analyzed through an exploratory data analysis stage, including class distribution, visual inspection, and pixel statistics. This analysis guides the preprocessing choices described in Sec.IV: normalization to $[0, 1]$ and light geometric data augmentation, without aggressive filtering, due to the risk of removing diagnostically relevant staining details.

The preprocessed images are then used in four parallel training pipelines, one for each architecture (Sec.V): a baseline CNN, a Squeeze-and-Excitation CNN, a patch-based decision fusion CNN, and a Spiking Neural Network. Each pipeline is trained independently using its own hyperparameter configuration and evaluated on the same held-out test set. This produces per-class metrics and confusion matrices, which are compared across architectures in Sec.VI.

An additional center-crop variant, motivated by a limitation identified in the patch-fusion results, is evaluated using the same pipeline as an ablation study.

IV. SIGNALS AND FEATURES

A. Data source and acquisition

BloodMNIST contains 17,092 microscopic images of single, healthy peripheral blood cells, acquired with a CellaVision DM96 analyzer at the Hospital Clinic of Barcelona [1]. The images belong to eight classes: basophil, eosinophil, erythroblast, immature granulocytes, lymphocyte, monocyte, neutrophil, and platelet. The original 360×363 RGB images were center-cropped to 200×200 pixels and resized to several standard resolutions (28, 64, 128, and 224 px) as part of the MedMNIST v2 benchmark [2]. This project uses the 64×64 resolution for the three CNN architectures and for the center-crop ablation, and also the native 28×28 resolution for a first SNN variant.

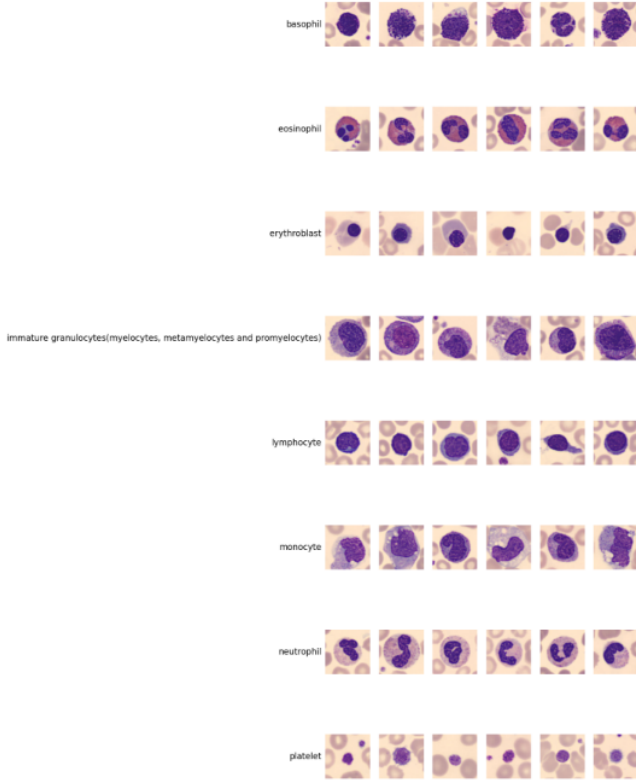


Fig. 2: Example training images for each of the 8 BloodM-NIST classes.

B. Preprocessing

Pixel values were normalized to the $[0, 1]$ range. During training, images were augmented with random horizontal flips, small random rotations (up to about 10% of 360°), and small random zooms (up to 10%), applied through Keras preprocessing layers that are active only during training. Apart from normalization and this light geometric augmentation, no filtering, contrast enhancement, or denoising was applied. This choice avoids removing staining-intensity detail that carries diagnostic information (Sec. VI reports an empirical class-imbalance check that supports the related decision not to apply class weighting).

C. Dataset statistics

Fig. 2 shows representative training images for each class. The eight classes are moderately imbalanced: neutrophils are the largest class in the training set (2,330 images, 19.5%), and lymphocytes are the smallest (849 images, 7.1%), giving an imbalance ratio of $2.74\times$. This is below the informal $10\times$ threshold often used as a trigger for corrective measures such as class weighting or oversampling, neither of which was applied here. Fig. 3 shows the full class distribution across all three data splits.

D. Train, validation, and test split

The official MedMNIST 70%/10%/20% split was used without changes: 11,959 training, 1,712 validation, and 3,421

test images. This keeps our results directly comparable with published MedMNIST v2 benchmarks.

V. LEARNING FRAMEWORK

Fig. 4 shows the shared convolutional backbone used by the baseline, Squeeze-and-Excitation, patch-fusion, and center-crop architectures, together with where the Squeeze-and-Excitation block is inserted. Fig. 5 shows the Spiking Neural Network architecture separately, since it follows a different computational paradigm and uses a different framework (PyTorch instead of TensorFlow).

A. Baseline CNN

The baseline network has three convolutional blocks (32, 64, and 128 filters). Each block applies a 3×3 convolution with ReLU activation, followed by batch normalization and 2×2 max pooling. After the three blocks, global average pooling reduces each channel to a single value, followed by a dense layer of 128 units with dropout ($p = 0.4$), and a final softmax layer over the 8 classes. The network has 111,240 trainable parameters. Even though the convolutional kernel size stays fixed at 3×3 in every block, stacking blocks lets the effective receptive field grow with depth: each block processes the feature maps produced by the previous block, not the raw pixels, and pooling shrinks the spatial size at every step. As a result, deeper layers represent larger and more complex patterns, even though no single filter ever changes size. This architecture is the reference point against which the benefit of attention, patch fusion, and the spiking paradigm is measured.

B. CNN with Squeeze-and-Excitation

A Squeeze-and-Excitation (SE) block [3] is added after batch normalization in each convolutional block. Given a feature map with C channels, the block works in three steps. First, global average pooling compresses each channel into a single number, producing a descriptor vector $z \in \mathbb{R}^C$ (*squeeze*). Second, a small bottleneck of two dense layers turns this descriptor into a set of per-channel importance weights (*excitation*):

$$s = \sigma(W_2 \delta(W_1 z)), \quad (1)$$

where $W_1 \in \mathbb{R}^{(C/r) \times C}$ and $W_2 \in \mathbb{R}^{C \times (C/r)}$ are the two learned weight matrices, $\delta(\cdot)$ is the ReLU activation, $\sigma(\cdot)$ is the sigmoid activation, and $r = 8$ is the reduction ratio that controls the bottleneck size. Third, the original feature map is rescaled channel-by-channel by these weights (*rescale*). This lets the network decide, separately for every input image, which channels are most useful, instead of always treating every channel as equally important. The resulting network has 116,868 trainable parameters.

C. Patch-Based Decision Fusion

Each 64×64 image is split into four non-overlapping 32×32 quadrants. The baseline architecture (Sec. V-A) is retrained on the resulting patches (47,836 training examples in total, each inheriting the label of its parent image). At test time, the four per-patch softmax outputs belonging to the same image are

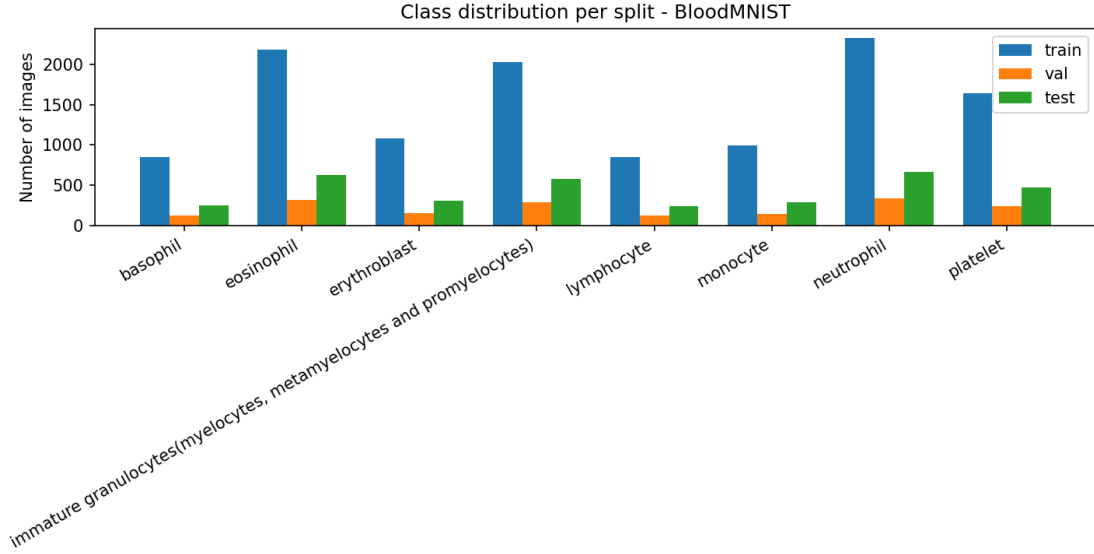


Fig. 3: Class distribution across the train, validation, and test splits.

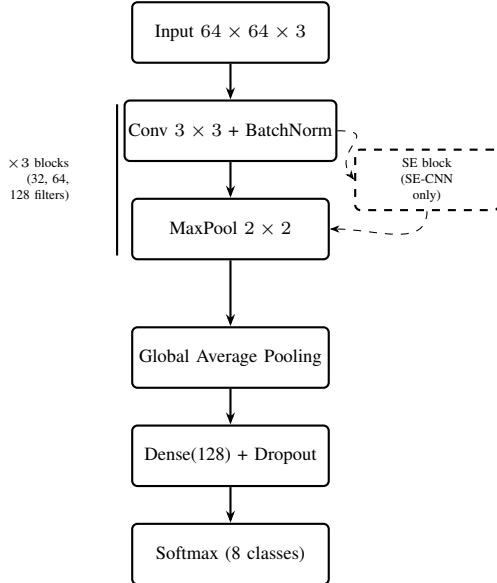


Fig. 4: Shared CNN backbone (baseline, patch-fusion, and center-crop architectures) with the Squeeze-and-Excitation insertion point marked.

averaged, and the class with the highest average probability is taken as the final prediction. This tests whether local, sub-image evidence, combined through averaging, improves over a single whole image view. Averaging also acts as an implicit test-time ensemble.

D. Spiking Neural Network

The Spiking Neural Network (SNN) follows a leaky integrate-and-fire (LIF) design adapted from the course laboratory. It has two convolution + max-pooling + leaky-neuron blocks (12 and 32 channels, 5×5 kernels, membrane decay

$\beta = 0.5$, 25 timesteps), followed by a fully-connected leaky output layer. The same static image is presented to the network at every timestep. The spiking (thresholding) operation is not differentiable. So training uses a surrogate gradient (a fast sigmoid) to approximate it during backpropagation. The predicted class is decoded through rate coding: the output neuron that fires the most spikes over all 25 timesteps is taken as the prediction. Two input resolutions were tested: 28×28 (14,648 parameters, matching the course laboratory as closely as possible) and 64×64 (53,816 parameters, matching the other three architectures for a fair comparison; see Sec. VI for the training adjustments this required). Per the course rules, PyTorch is used only for this architecture; every other architecture uses TensorFlow/Keras.

E. Center-Crop CNN (Ablation)

BloodMNIST cells are centered in the frame, since the dataset itself was built using a center crop of the original photographs [2]. This means the quadrant split used in patch fusion (Sec. V-C) always cuts the cell across a patch boundary. To test whether this matters, this ablation instead crops a single 48×48 region (75% of the image) from the exact center of each 64×64 image, removing an even border of background on every side. It then retraining the baseline architecture with no other changes. This isolates one specific factor: whether removing background alone explains fusion’s advantage over the baseline. Patch fusion also benefits from a fourfold increase in training data and from test-time averaging. The center-crop model has neither of these two mechanisms.

VI. RESULTS

A. Overall comparison

Tab. 1 summarizes the test-set accuracy of all seven model variants trained in this project. Among the four required

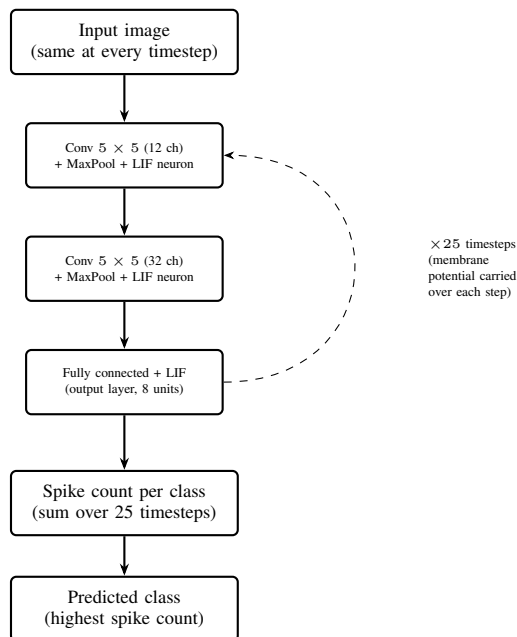


Fig. 5: Spiking Neural Network architecture, showing the timestep loop and the leaky integrate-and-fire neurons.

TABLE 1: Test accuracy and parameter count for all model variants.

Model	Accuracy	Params
Baseline CNN	91.58%	111,240
SE-CNN (first attempt)*	89.27%	116,868
SE-CNN (tuned)	96.99%	116,868
Patch fusion CNN	96.87%	111,240
SNN (28×28)	64.45%	14,648
SNN (64×64)	87.26%	53,816
Center-crop CNN (ablation)	96.84%	111,240

architectures, the tuned SE-CNN reaches the highest accuracy, closely followed by patch fusion; the baseline CNN trails both by a wider margin (91.58%). The SNN uses roughly half the parameters of the CNN variants, yet still comes within 4 points of the baseline CNN - a reasonable result given its fundamentally different, event-driven computational paradigm (Sec. V-D) and the much shorter time available to search for good hyperparameters compared to the CNNs.

B. Training instabilities: why two results needed correction

Two of the results above needed a second, corrected training run, and both cases are worth explaining in detail, because they show how an architectural change can affect training stability, not just accuracy.

The first SE-CNN training run used the same settings that trained the baseline stably (learning rate 10^{-3} , EarlyStopping patience of 6 epochs), but it showed clear oscillation in validation accuracy during the first several epochs - for example jumping from 13.7% up to 38.4%, then to 74.7%, 76.2%, 75.0%, 90.4%, then dropping sharply to 18.6% before recovering to 37.6%. This is a symptom of unstable training.

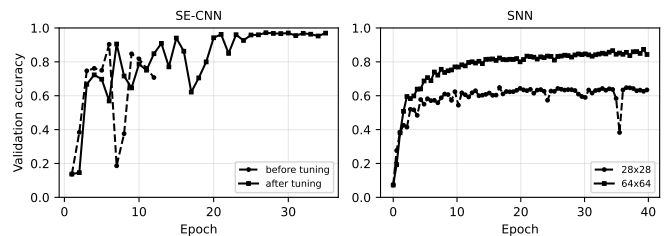


Fig. 6: Left: SE-CNN validation accuracy before and after tuning the learning rate and patience. Right: SNN validation accuracy at the two input resolutions tested (28×28 vs. 64×64).

Lowering the learning rate to 5×10^{-4} and raising the patience to 10 epochs gave smoother, more consistent convergence, reaching 96.99% accuracy.

A similar problem was observed in an earlier training session with the SNN at 64×64 : using the same learning rate (10^{-2}) that trained the 28×28 version normally caused training to collapse completely, with the network reaching only 19.47% accuracy and predicting almost always the same class. The likely cause was the size of the final layer, which is much larger at 64×64 (5,408 inputs) than at 28×28 (512 inputs), making the same learning rate produce larger, destabilizing weight updates. Lowering the learning rate to 10^{-3} solved the problem in that session. The run included in the final submitted notebook uses this corrected learning rate throughout and reaches 87.26% accuracy.

Both cases show the same general lesson: adding a mechanism (attention) or changing part of the network (a larger final layer) can shift what counts as a stable learning rate or stopping criterion, even when everything else in the training setup stays the same. This is an important thing to keep in mind whenever different architectures are compared under one shared hyperparameter configuration.

C. Per-class performance and the imbalance check

Platelet and eosinophil are classified almost perfectly (F1-score = 1.00) by the best model (tuned SE-CNN). This matches their distinctive appearance: platelets are small cell fragments with no nucleus, while eosinophils have orange-pink granules instead of the purple granules seen in other cell types. Immature granulocytes have the lowest recall (0.88), and neutrophils and monocytes the lowest precision (0.92 each). Tab. 2 shows that the most common error is between immature granulocytes and neutrophils (52 cases), consistent with them sharing the same cell lineage and being morphologically similar (Sec. IV).

For the best-performing model (tuned SE-CNN), the macro-averaged and weighted-averaged F1-scores are both 0.97. Macro averaging treats every class equally, while weighted averaging gives more weight to larger classes; if the moderate class imbalance described in Sec. IV were inflating the overall accuracy by rewarding good performance mainly on the larger classes, the macro F1-score would be clearly lower than the weighted one. Since the two values match, this confirms that

TABLE 2: Confusion matrix for the best model (SE-CNN, tuned). Rows: true class; columns: predicted class.

True \ Pred	Baso	Eosi	Eryt	ImGr	Lymp	Mono	Neut	Plat
Baso	241	0	0	1	0	1	1	0
Eosi	0	624	0	0	0	0	0	0
Eryt	0	0	306	1	1	2	1	0
ImGr	5	0	2	510	0	10	52	0
Lymp	0	0	5	0	230	8	0	0
Mono	0	0	0	4	1	279	0	0
Neut	1	1	0	3	0	3	658	0
Plat	0	0	0	0	0	0	0	470

the reported accuracy is not an artifact of class imbalance, supporting the earlier decision not to apply class weighting.

D. The center-crop ablation and the limits of a single training run

In this final training run, the tuned SE-CNN (96.99%) is the best model overall, narrowly ahead of patch fusion (96.87%) and the center-crop CNN (96.84%) - all three within 0.15 points of each other. This is close enough that the ranking can change just because of randomness between runs: neither the CNN scripts nor Keras’s data augmentation and shuffling used a fixed random seed, so the starting weights, the augmentation choices, and the order of the training data are all slightly different every time the code runs, even with identical settings.

Because of this, it is not possible to confidently say what really causes patch fusion’s advantage over the plain baseline, based on a single run of each model. Two explanations are still possible: removing background (what the center crop tests) and combining several predictions at test time (only patch fusion does this). It is difficult to tell which one matters more without running both experiments several times with different random seeds and comparing the average accuracy, not just one number each. This is itself a useful finding: when models are this close in accuracy, one run is not enough to rank them with confidence. Future work extending this project should report results averaged over several seeds before treating either explanation as proven.

E. Complexity and efficiency

Tab. 3 reports the training time for each model. All three CNN variants have a similar number of parameters (111,240-116,868), so their accuracy differences come from architectural design choices rather than from raw model size. The SNN has fewer parameters than any CNN, but it needs 25 forward passes per image (one per timestep), which makes it noticeably slower to train per epoch than any CNN despite being smaller - a trade-off that would not be visible by looking at the parameter count alone.

VII. CONCLUDING REMARKS

A. Summary and observations

This work compared four architecturally different approaches to blood cell classification on BloodMNIST, plus one additional ablation experiment motivated by a limitation found during the project itself. Two observations go beyond the raw accuracy numbers reported in Sec. VI. First, adding

TABLE 3: Training time per model (single Kaggle T4/P100 GPU session).

Model	Training time
Baseline CNN	1m 30s
SE-CNN (tuned)	3m 25s
Patch fusion CNN	3m 38s
Center-crop CNN (ablation)	1m 1s
SNN (28×28 , 40 epochs)	5m 20s
SNN (64×64 , 40 epochs)	13m 5s

an architectural component can shift which training hyperparameters actually work, even when everything else stays the same. I saw this clearly for the SE-CNN, and a similar pattern was observed in an earlier session with the SNN (Sec. VI-B). Second, comparing architectures through an ablation study (the center-crop experiment) showed that two models close in accuracy can rank differently between separate training runs, since neither model’s training used a fixed random seed. This is a useful reminder that a single training run is not always enough to confidently explain why one architecture does better than another, especially when the accuracy gap is small.

B. Limitations and future work

The SNN was tested with only one leaky-neuron configuration and only rate coding; trying different timestep counts, decay values (β), or coding schemes was outside the time available for this project. The center-crop fraction (75%) was also not varied, so it remains unclear whether a different crop size would give a more consistent advantage over patch fusion. Combining Squeeze-and-Excitation with the center-cropped input was not tested, even though it could plausibly combine both sources of improvement found in this work. Finally, this project highlights the value of repeating key comparisons across multiple random seeds, particularly for models with similar accuracy such as the tuned SE-CNN, patch fusion, and the center-crop CNN, an addition left for future work.

C. What I learned

Beyond the specific architectures, this project taught me a more general lesson: a low or unexpected result (as with the first SE-CNN run, and with the SNN in an earlier session) is often caused by the training configuration, not by the architecture itself. The two are only told apart by changing one hyperparameter at a time and re-running the experiment, instead of accepting the first number obtained. I also learned that even a carefully designed ablation study, like the center-crop experiment, can give a different answer on a different run when the two models being compared are close in accuracy, a reminder that a single comparison, however well designed, does not always tell the full story on its own.

D. Difficulties encountered

The main difficulties were diagnostic rather than conceptual. Two architectures (the SE-CNN and the SNN at higher resolution) gave results that were, at first, hard to tell apart from genuine architectural weaknesses, and required targeted,

single-variable follow-up experiments (Sec. VI-B) to understand what was really happening.

REFERENCES

- [1] A. Acevedo, A. Merino, S. Alf  rez,   . Molina, L. Bold  , and J. Rodellar, “A dataset of microscopic peripheral blood cell images for development of automatic recognition systems,” *Data in Brief*, vol. 30, p. 105474, 2020.
- [2] J. Yang, R. Shi, D. Wei, Z. Liu, L. Zhao, B. Ke, H. Pfister, and B. Ni, “MedMNIST v2 – a large-scale lightweight benchmark for 2d and 3d biomedical image classification,” *Scientific Data*, vol. 10, no. 1, p. 41, 2023.
- [3] J. Hu, L. Shen, and G. Sun, “Squeeze-and-excitation networks,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, (Salt Lake City, UT, USA), pp. 7132–7141, June 2018.
- [4] A. Acevedo, S. Alf  rez, A. Merino, L. Puigv  , and J. Rodellar, “Recognition of peripheral blood cell images using convolutional neural networks,” *Computer Methods and Programs in Biomedicine*, vol. 180, p. 105020, 2019.